Solution du DS 1

CPES « Sciences des données, arts et cultures » : Introduction à l'algorithmique

Antoine GAUQUIER antoine.gauquier@ens.fr
Pierre SENELLART pierre@senellart.com

14 novembre 2023

A. Mot le plus fréquent

Question 1. (1 point) La complexité d'accès à un élément arbitraire d'un dictionnaire Python est en $O(1)$ (ou, de manière équivalente, en $\Theta(1)$).

Question 2. (3 points)

```
def clef_plus_grande_valeur(d):
    plus_grande_valeur = None
    resultat = None

    for clef in d:
        if plus_grande_valeur == None or plus_grande_valeur<d[clef]:
            plus_grande_valeur=d[clef]
            resultat=clef

    return resultat
```

Question 3. (2 points) La fonction `clef_plus_grande_valeur` commence par des initialisations (en $O(1)$) puis une boucle parcourant l'ensemble des éléments du dictionnaire, qui effectue un test (avec accès à un élément du dictionnaire, en O(1)) puis selon le résultat du test fait deux affectations (avec accès à un élément du dictionnaire, en O(1)). Si le nombre d'éléments du dictionnaire est n, la fonction est donc en $\Theta(n)$ (n itérations de la boucle, avec un temps de $O(1)$ par itération).

Question 4. (2 points)

```
def mot_plus_frequent(l):
    occurrences = {}
    for mot in l:
        if mot in occurrences:
            occurrences[mot] += 1
        else:
            occurrences[mot] = 1
    return clef_plus_grande_valeur(occurrences)
```

Question 5. (2 points) Appelons n le nombre de mots dans la liste. La fonction `mot_plus_frequent` commence par créer un dictionnaire vide, puis fait n itérations d'une boucle dont le contenu (test, accès à des valeurs d'un dictionnaire) se fait en $O(1)$. Ensuite, elle exécute la fonction `clef_plus_grande_valeur` sur un dictionnaire formé des occurrences (dont le nombre d'éléments est donc au plus n). On obtient donc une complexité de $\Theta(n)$ pour la boucle et $\Theta(n')$ pour l'appel à `clef_plus_grande_valeur` (avec n le nombre d'éléments du dictionnaire, et $n' < n$) soit $\Theta(n)$ pour l'ensemble de la fonction.

B. Chaînes bien parenthésées

Question 6. (1 point) On peut prendre la fonction f définie par : $f(0) = 9$, $f(1) = 4$, $f(2) = 3$, $f(5) = 8$, $f(6) = 7$, $f(10) = 11$.

Question 7. (3 points)

```
def est_bien_parenthesee(chaine):
    pile = []
    correspondance={"[": "]", "(": ")"}
    for c in chaine:
        if c in correspondance:
            pile.append(c)
        else:
            if len(pile)>0 and correspondance[pile[len(pile)-1]]==c:
                pile.pop()
            else:
                return False
    return len(pile)==0
```

Question 8. (2 points) La fonction `est_bien_parenthesee` parcourt, après des initialisations en $O(1)$, chaque caractère de la chaîne et pour chacun effectue des opérations (tests, comparaisons, accès à un dictionnaire, accès à la longueur d'une liste Python, accès à un élément arbitraire d'une liste Python, ainsi qu'ajout d'un élément à la fin et suppression du dernier élément d'une liste Python) qui snt toutes en $O(1)$. Noter que pour l'ajout et la suppression du dernier élément d'une liste Python, c'est une complexité *amortie*, les listes Python étant des tableaux dynamiques. La complexité est donc en $\Theta(n)$ où n est la longueur de la chaîne.

Question 9. (2 points)

```
class Chaine:
    def __init__(self, chaine):
        self.__c=chaine

    def taille(self):
        return len(self.__c)

    def est_bien_parenthesee(self):
        # reprendre la fonction est_bien_parenthesee, mais
        # remplacer chaine par self.__c
```

Question 10. (2 points) Il suffit d'utiliser un compteur des nombres de parenthèses ouvrantes et fermantes rencontrées, au lieu d'une pile. Par exemple, en Python :

```
def est_bien_parenthesee_seulement_parentheses(chaine):
    compteur = 0
    for c in chaine:
        if c=='(':
            compteur += 1
        elif c==')' and compteur>0:
            compteur -= 1
        else:
            return False
    return compteur==0
```